

The background is a light purple map of a city street network. Overlaid on this map is a graph structure consisting of black lines representing edges and small white circles representing vertices. The graph follows the main roads and intersections of the city streets.

Teoria dos Grafos

Aula 5-1:
Algoritmo em grafos

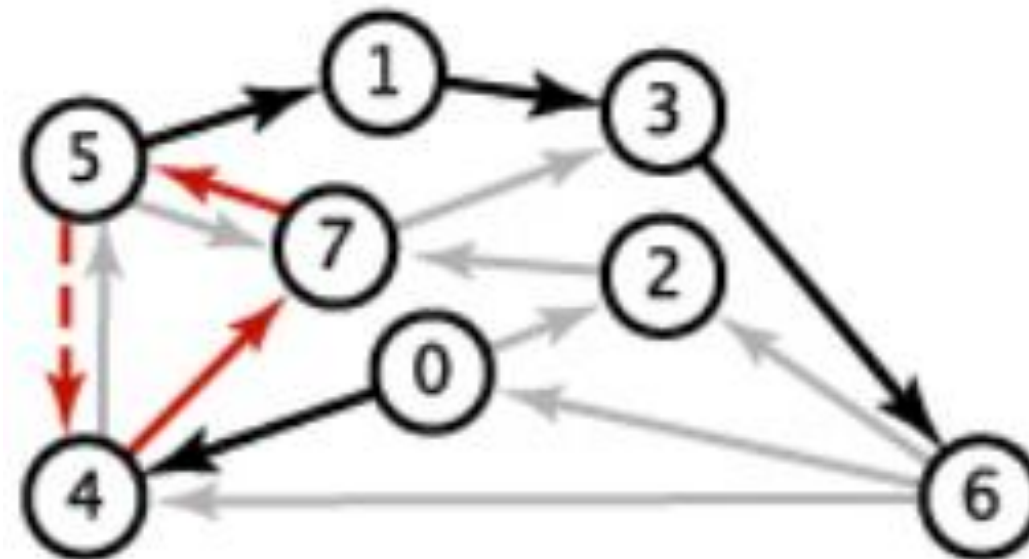
Profa. Kátia Fernandes

Algoritmos em Grafos

Ciclo Negativo: Seja um grafo com custos nas arestas, um ciclo é negativo se seu custo for negativo.

A tabela a seguir mostra um grafo com custos nas arestas. O caminho **4-7-5-4** é um ciclo negativo. Seu custo é -1.

0-2	0-4	1-3	2-7	3-6	4-5	4-7	5-1	5-4	5-7	6-0	6-2	6-4	7-3	7-5
26	38	29	34	52	35	37	32	-66	28	58	40	93	39	28



Algoritmos em Grafos

Caminho mínimo entre todos os pares de vértices:

Algoritmo de Floyd - Warshall



Algoritmo de Floyd -Warshall

- ▶ Seja $D = (V, A)$ um grafo dirigido, *possivelmente com custos negativos nas arestas, mas sem ciclos negativos*. O algoritmo de Floyd-Warshall (1959, 1962) determina *a distância entre cada par de vértices de D* .
- ▶ O algoritmo mantém uma distância parcial para cada par de vértices u e v , e vai melhorando este valor até que o valor correto seja encontrado.



Algoritmo de Floyd -Warshall

Para cada par de vértices i e j , e um inteiro k , um caminho mínimo entre i e j que utiliza somente vértices internos do conjunto $\{1, 2, \dots, k\}$ pode ser:

- ▶ um caminho que utiliza somente vértices internos do conjunto $\{1, 2, \dots, k - 1\}$; ou
- ▶ um caminho de i a k concatenado a um caminho de k a j , ambos utilizando somente vértices internos do conjunto $\{1, 2, \dots, k - 1\}$.



Algoritmo de Floyd -Warshall

- ▶ Seja $f(i, j, k)$ a distância entre os vértices i e j utilizando somente vértices internos do conjunto $\{1, 2, \dots, k\}$.
- ▶ Os valores iniciais de f são $f(i, j, 0) = c(i, j)$.
- ▶ A fórmula geral é:
$$f(i, j, k) = \min\{f(i, j, k-1), f(i, k, k-1) + f(k, j, k-1)\}$$
- ▶ O que queremos é calcular o valor $f(i, j, n)$.



Algoritmo de Floyd -Warshall

1. Para $i = 1, 2, \dots, n$ faça

$$d(i, i) \leftarrow 0;$$

2. Para cada aresta (i, j) faça

$$d(i, j) \leftarrow c(i, j);$$

3. Para $k = 1$ até n faça

Para $i = 1$ até n faça

Para $j = 1$ até n faça

Se $d(i, k) + d(k, j) < d(i, j)$ então

$$d(i, j) \leftarrow d(i, k) + d(k, j)$$

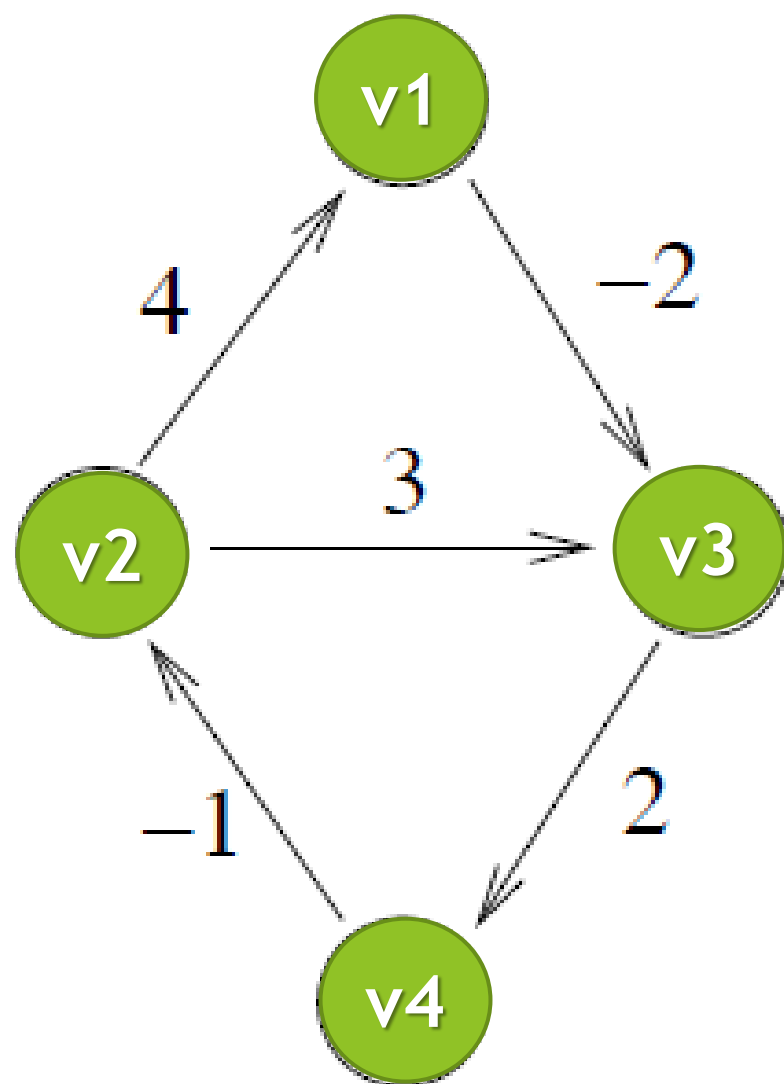


Algoritmo de Floyd -Warshall

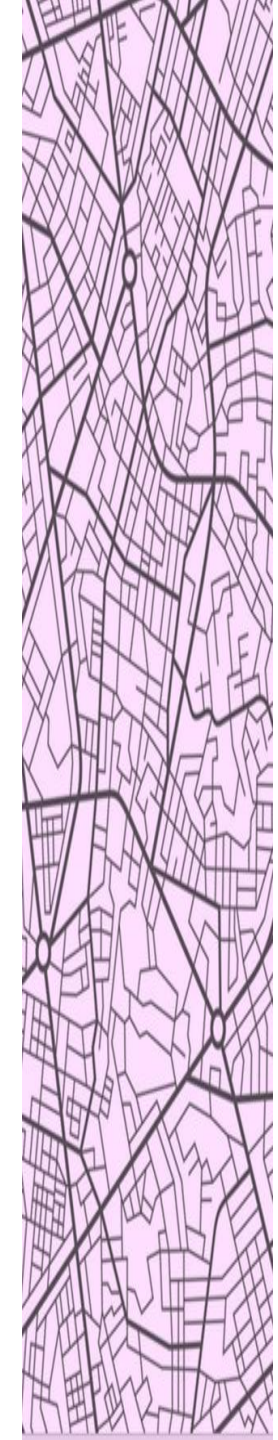
- ▶ É um algoritmo que resolve o problema de calcular o caminho mais curto entre todos os pares de vértices em um grafo dirigido e valorado (com peso).



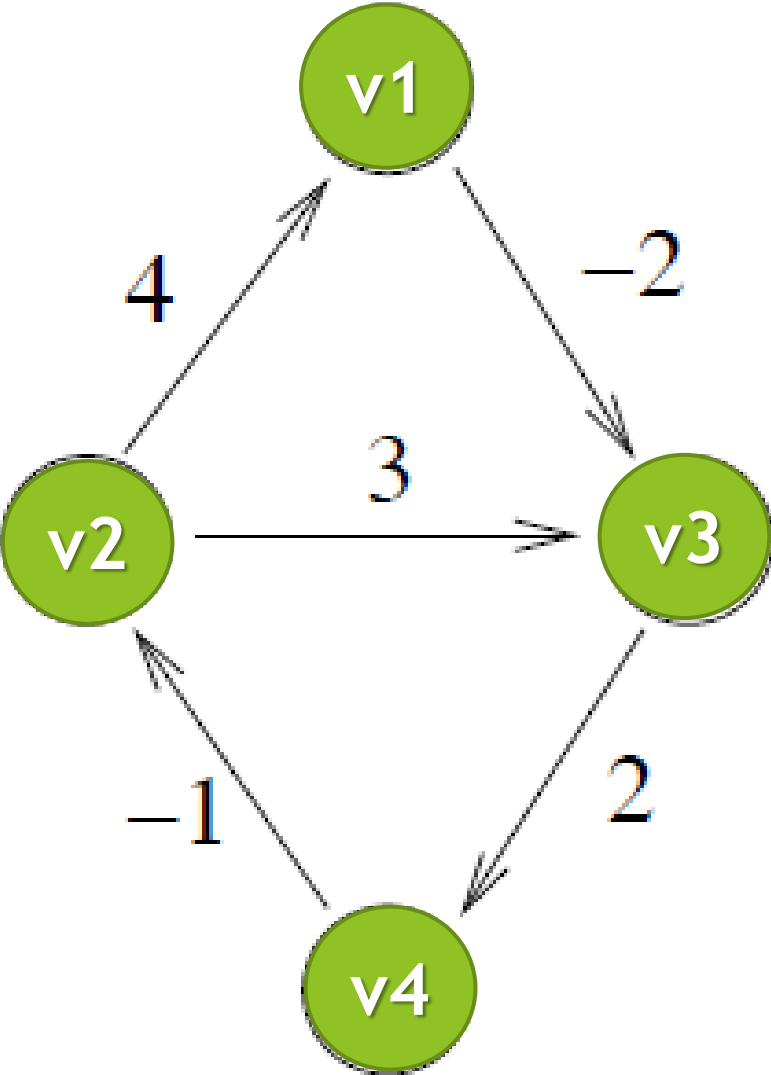
Algoritmo de Floyd -Warshall



	v1	v2	v3	v4
v1				
v2				
v3				
v4				

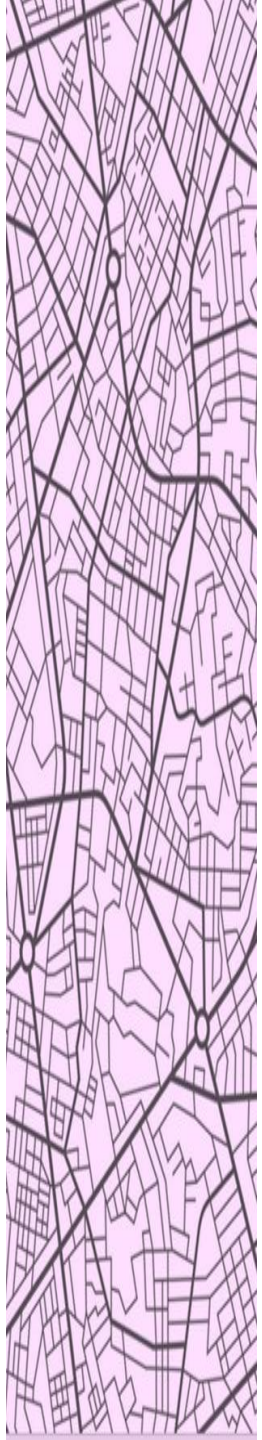


Algoritmo de Floyd -Warshall

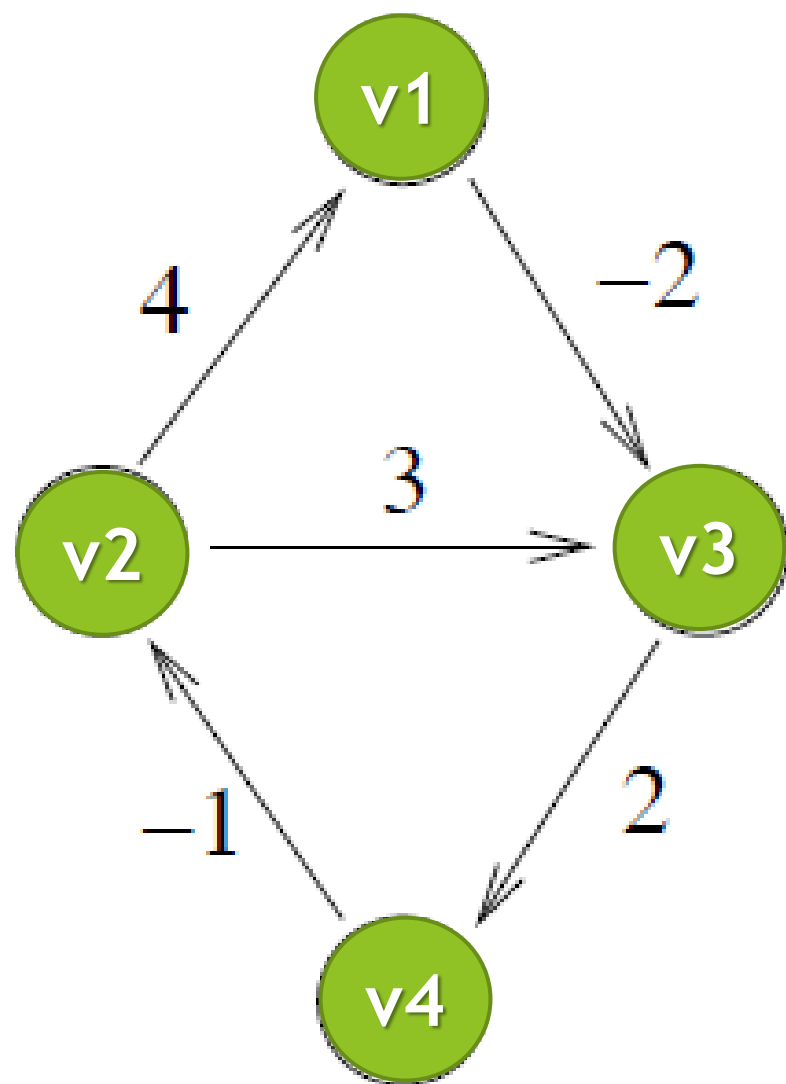


	v1	v2	v3	v4
v1	0			
v2		0		
v3			0	
v4				0

k= 0

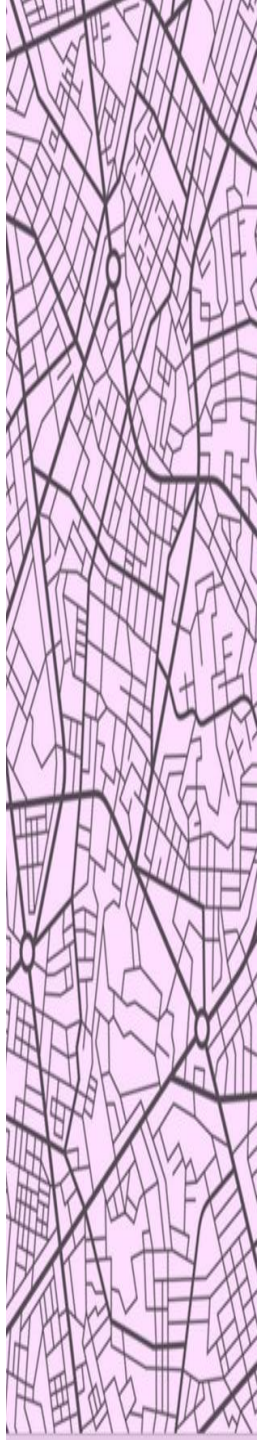


Algoritmo de Floyd -Warshall

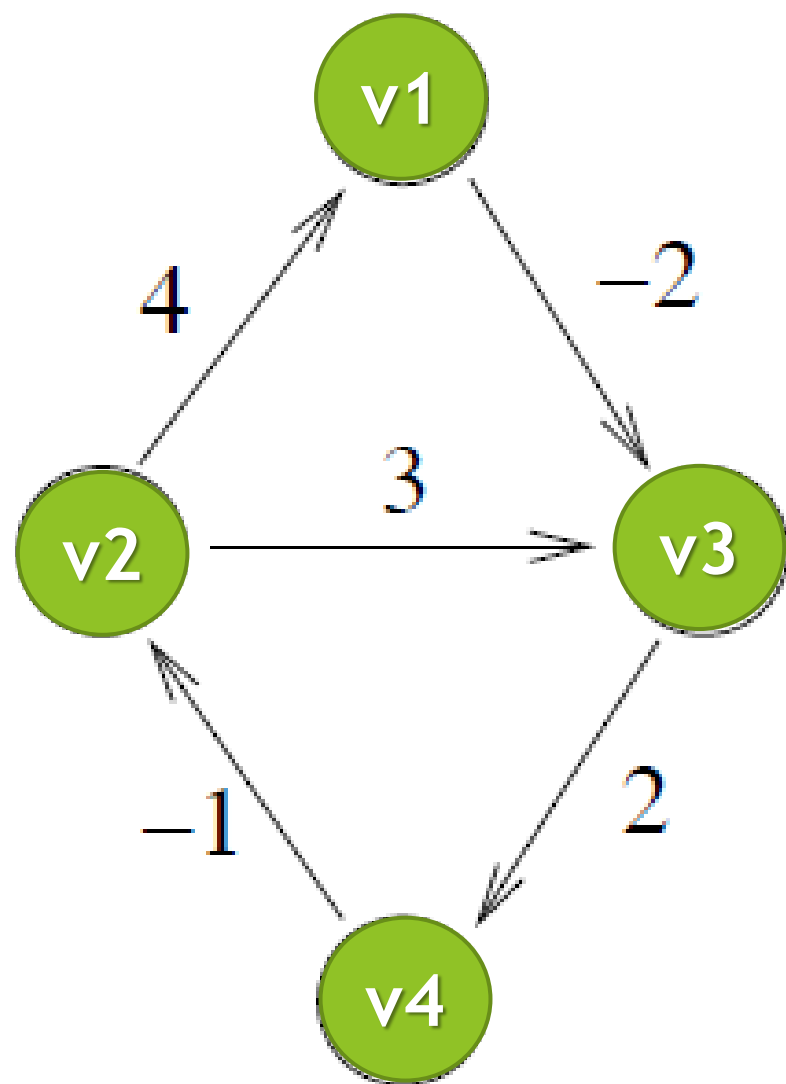


	v1	v2	v3	v4
v1	0		-2	
v2	4	0	3	
v3			0	2
v4		-1		0

k= 0

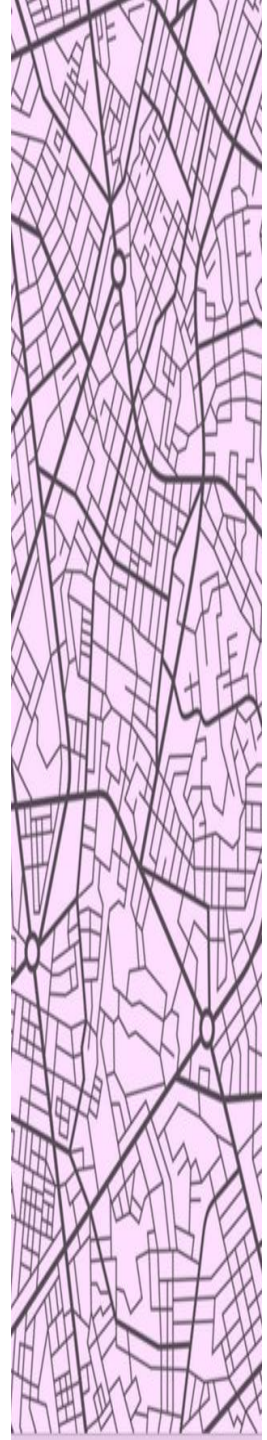


Algoritmo de Floyd -Warshall

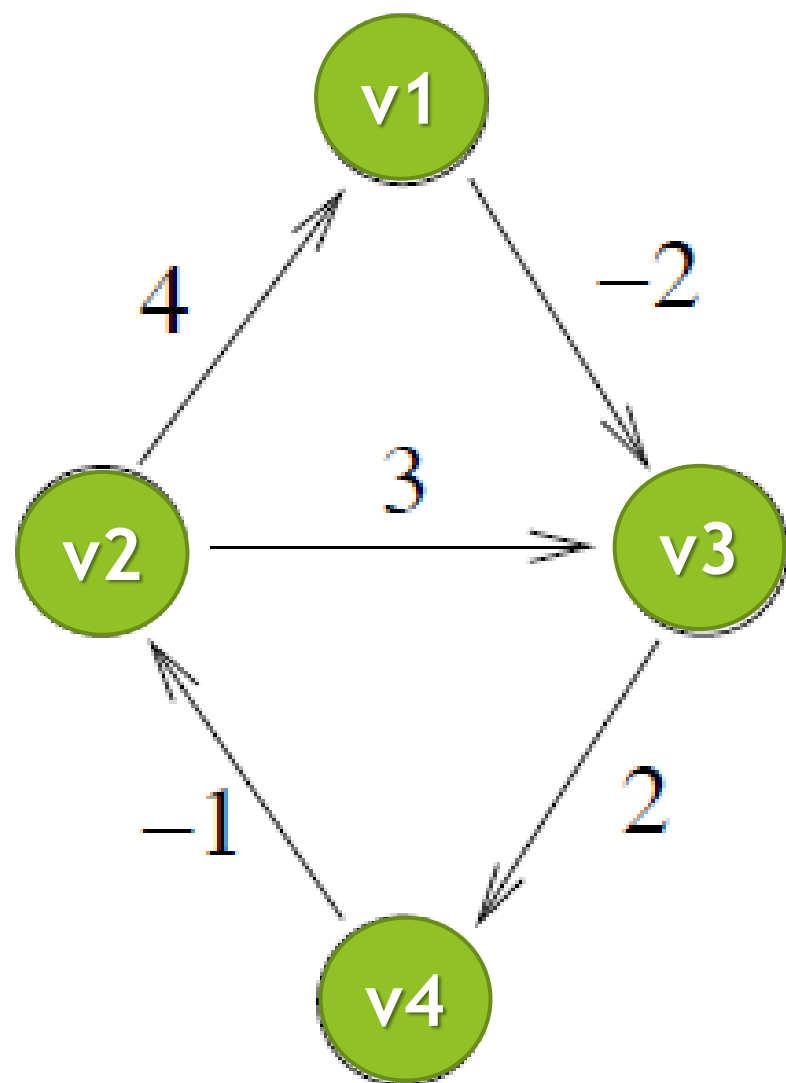


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	3	∞
v3	∞	∞	0	2
v4	∞	-1	∞	0

k= 0

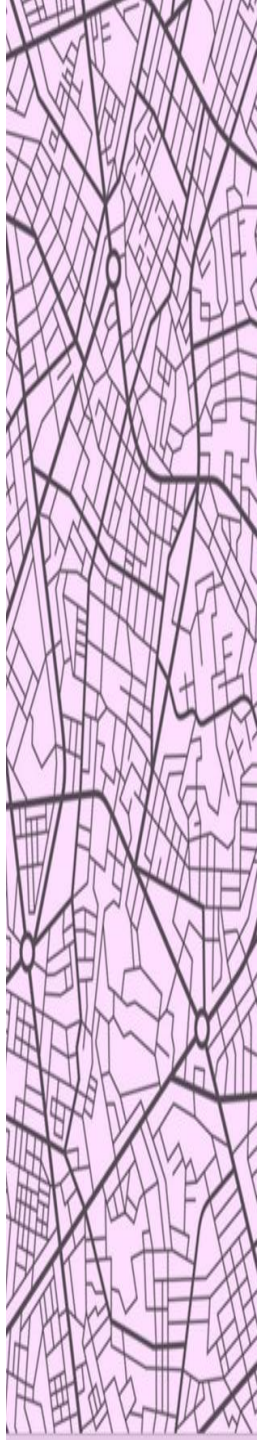


Algoritmo de Floyd -Warshall

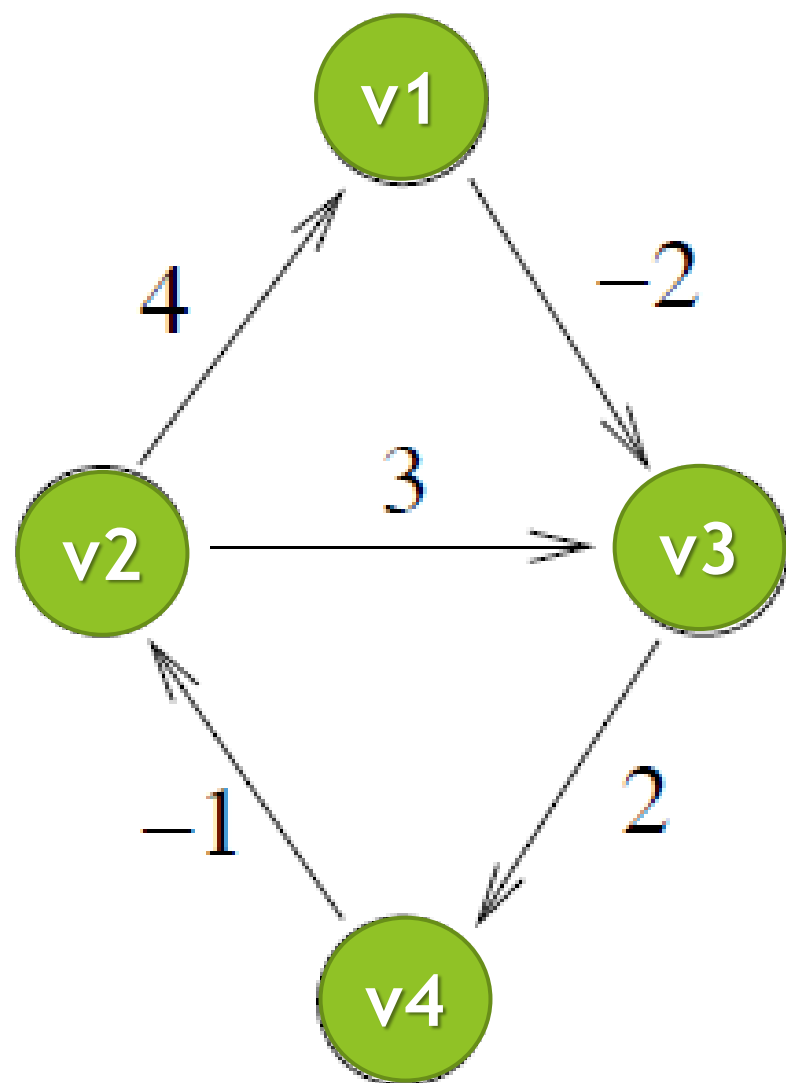


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	3	∞
v3	∞	∞	0	2
v4	∞	-1	∞	0

k= 1

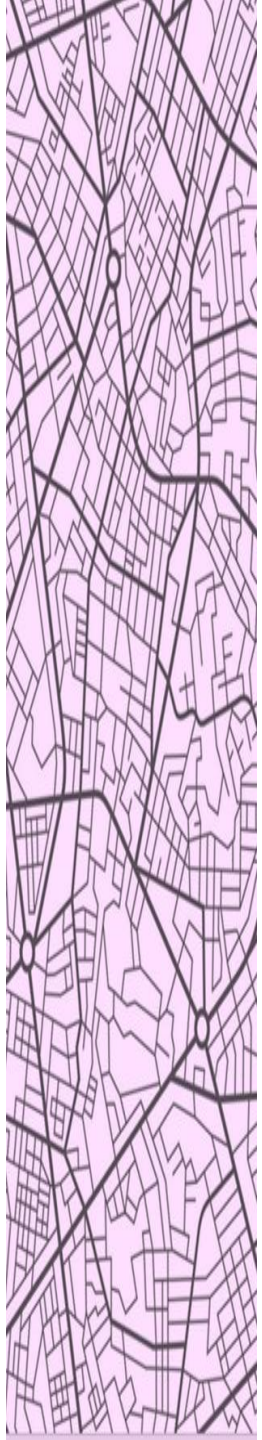


Algoritmo de Floyd -Warshall

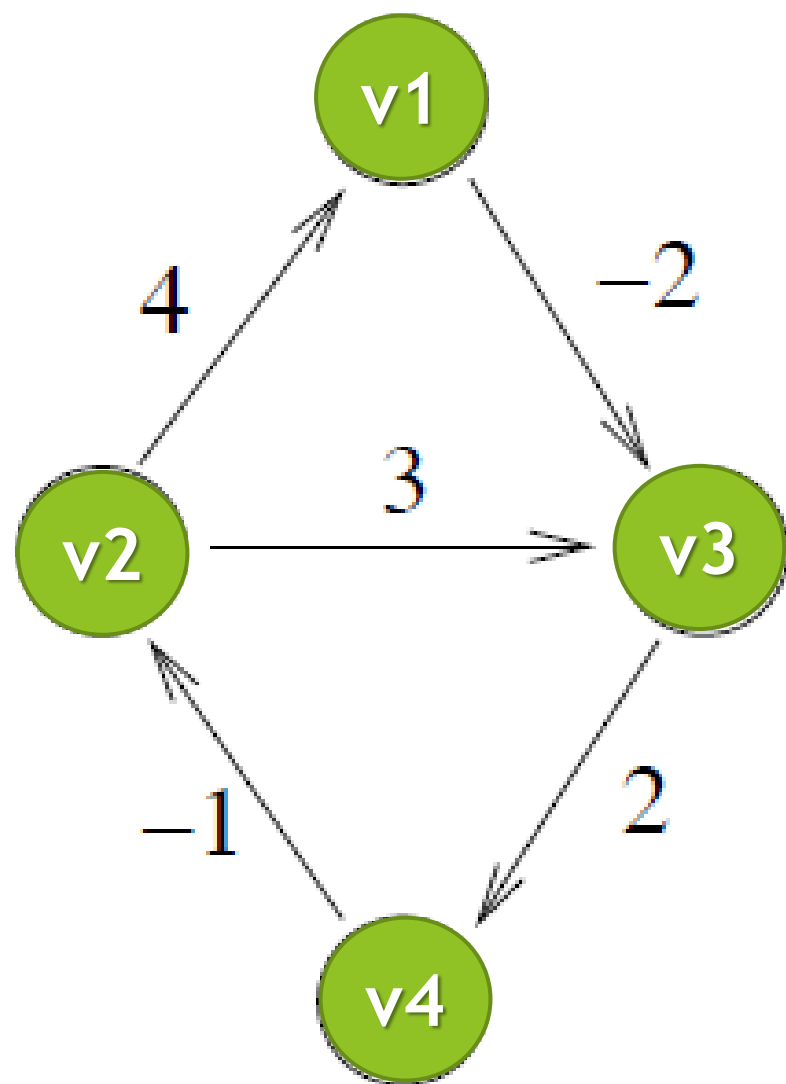


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	2	∞
v3	∞	∞	0	2
v4	∞	-1	∞	0

k= 1

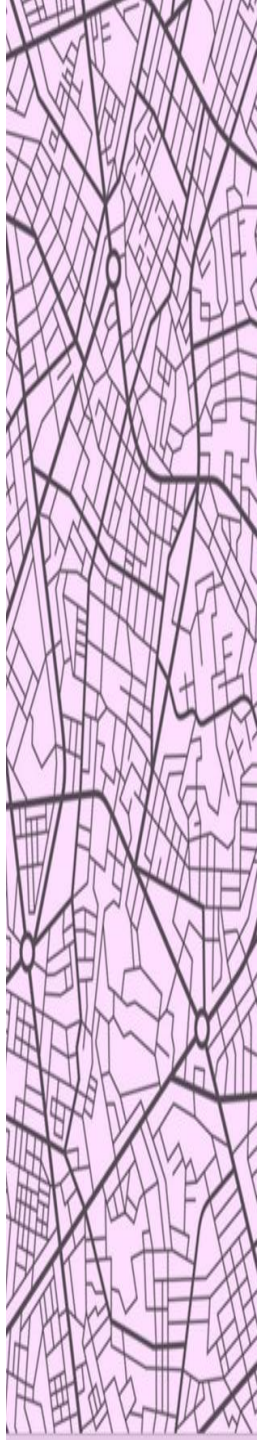


Algoritmo de Floyd -Warshall

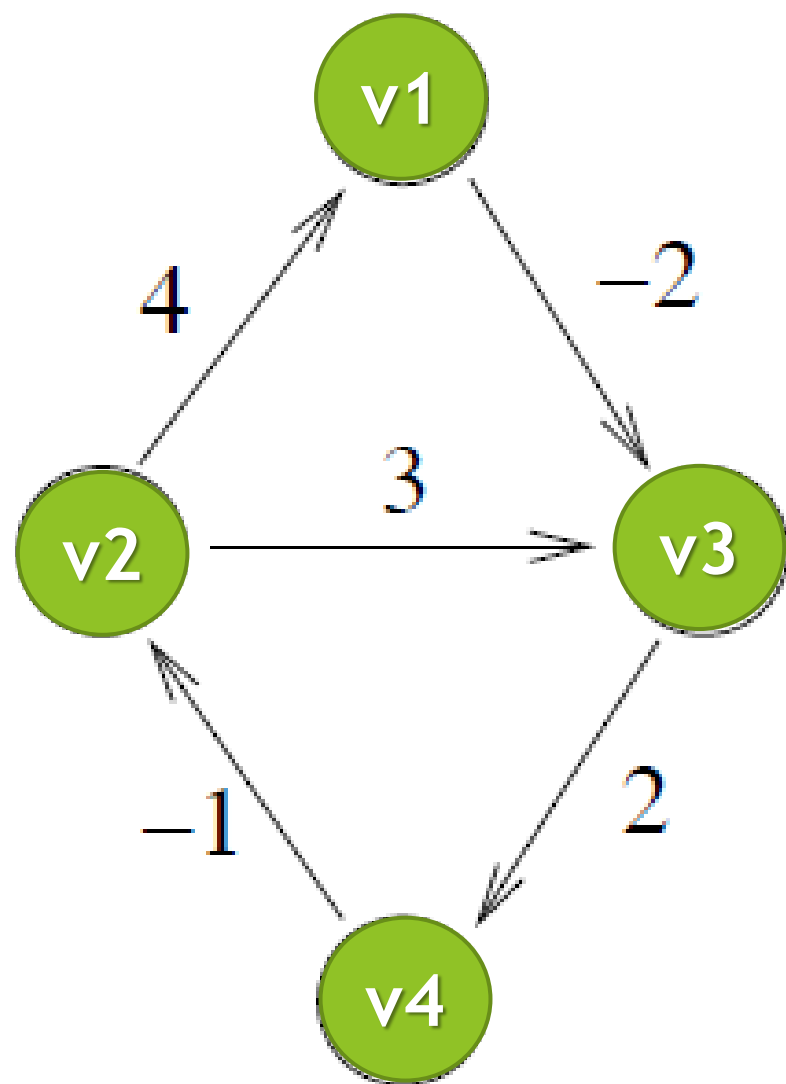


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	2	∞
v3	∞	∞	0	2
v4	∞	-1	∞	0

k= 2

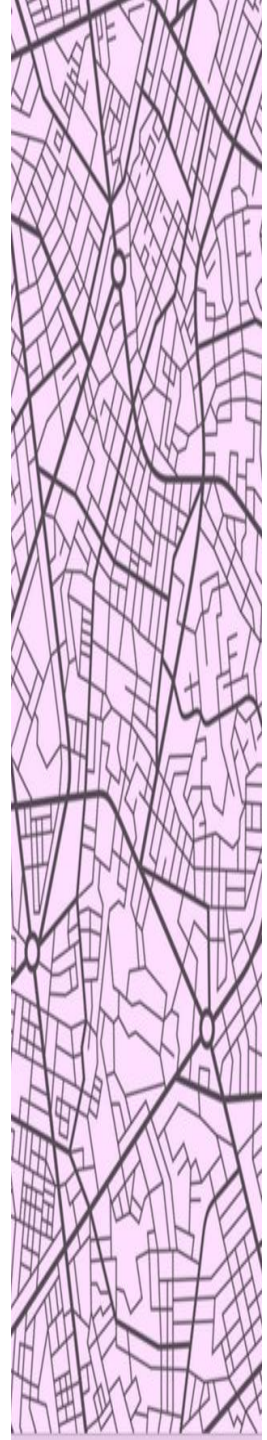


Algoritmo de Floyd -Warshall

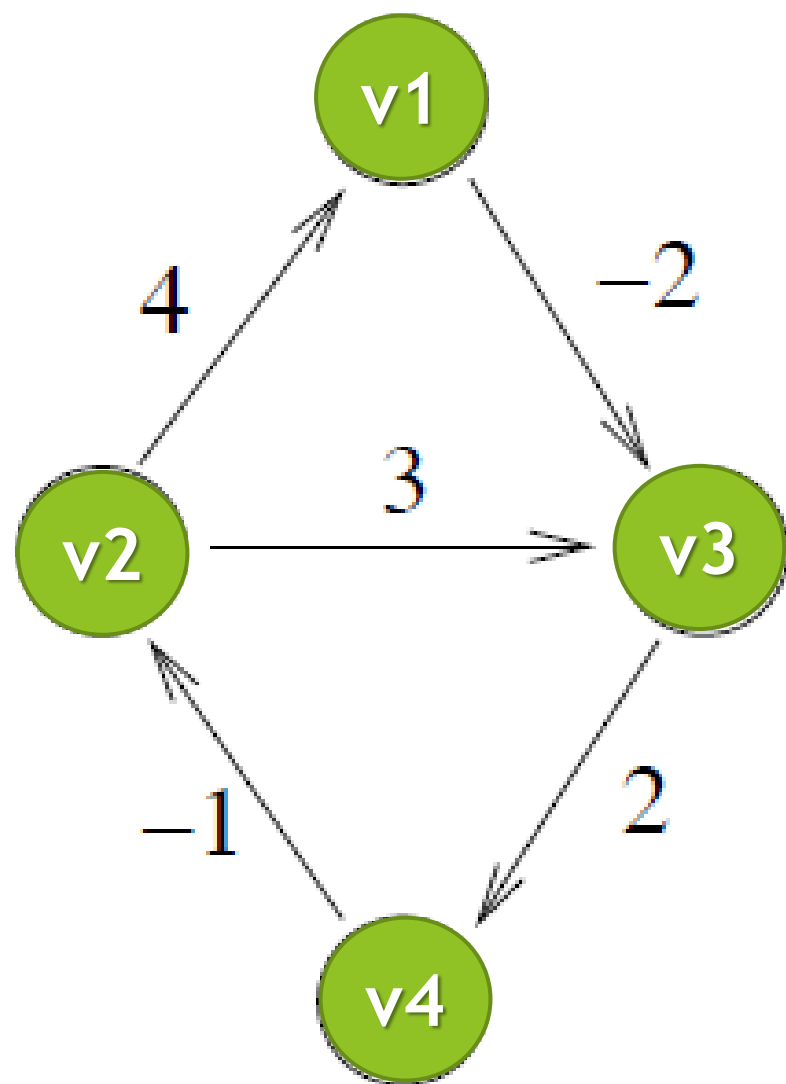


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	2	∞
v3	∞	∞	0	2
v4	3	-1	1	0

k= 2

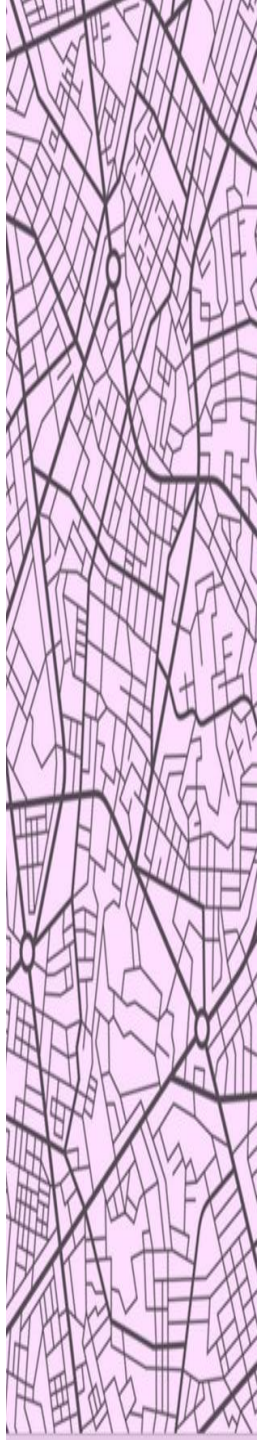


Algoritmo de Floyd -Warshall

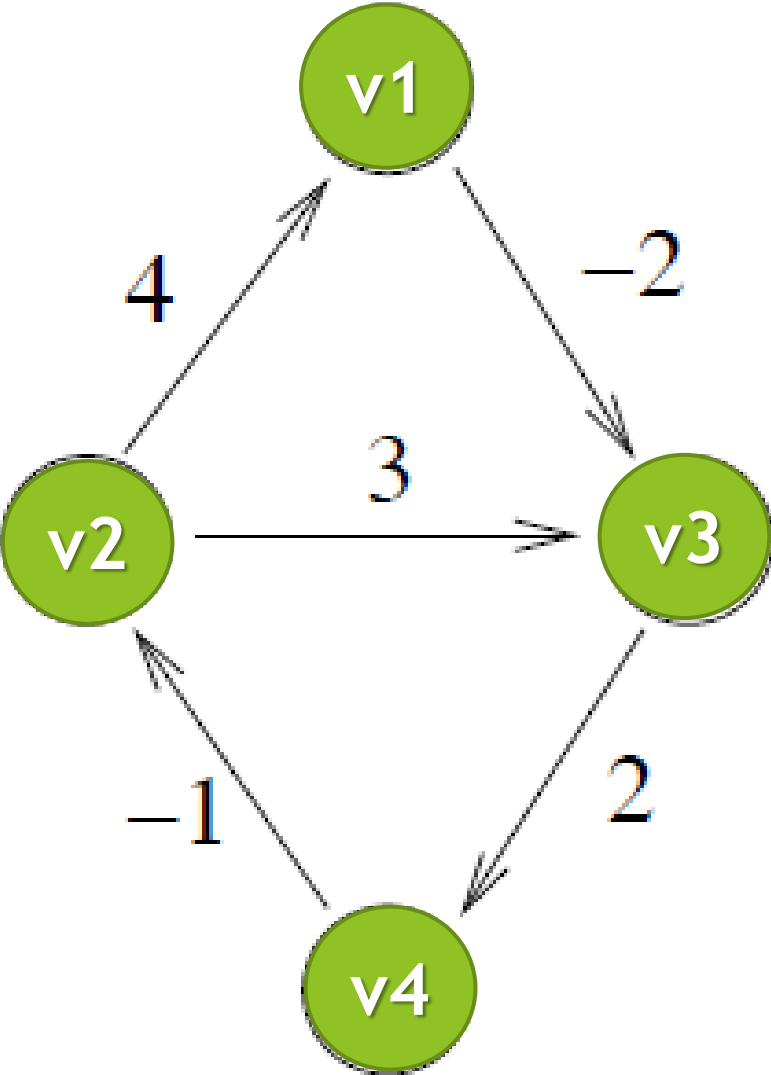


	v1	v2	v3	v4
v1	0	∞	-2	∞
v2	4	0	2	∞
v3	∞	∞	0	2
v4	3	-1	1	0

k= 3

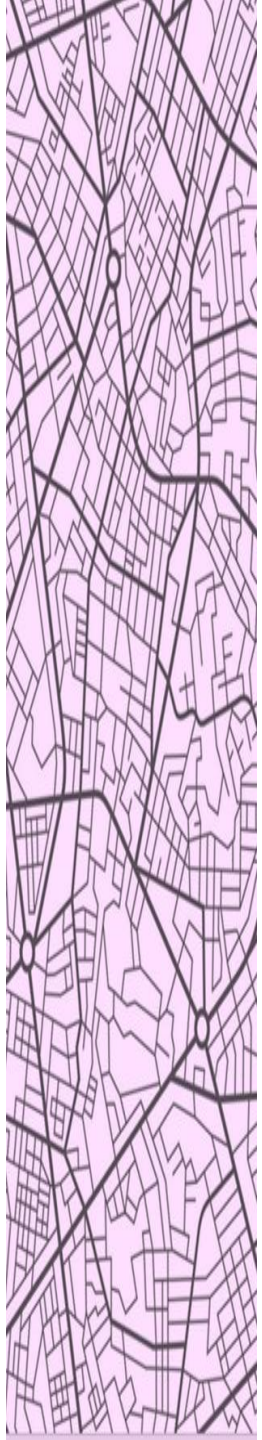


Algoritmo de Floyd -Warshall

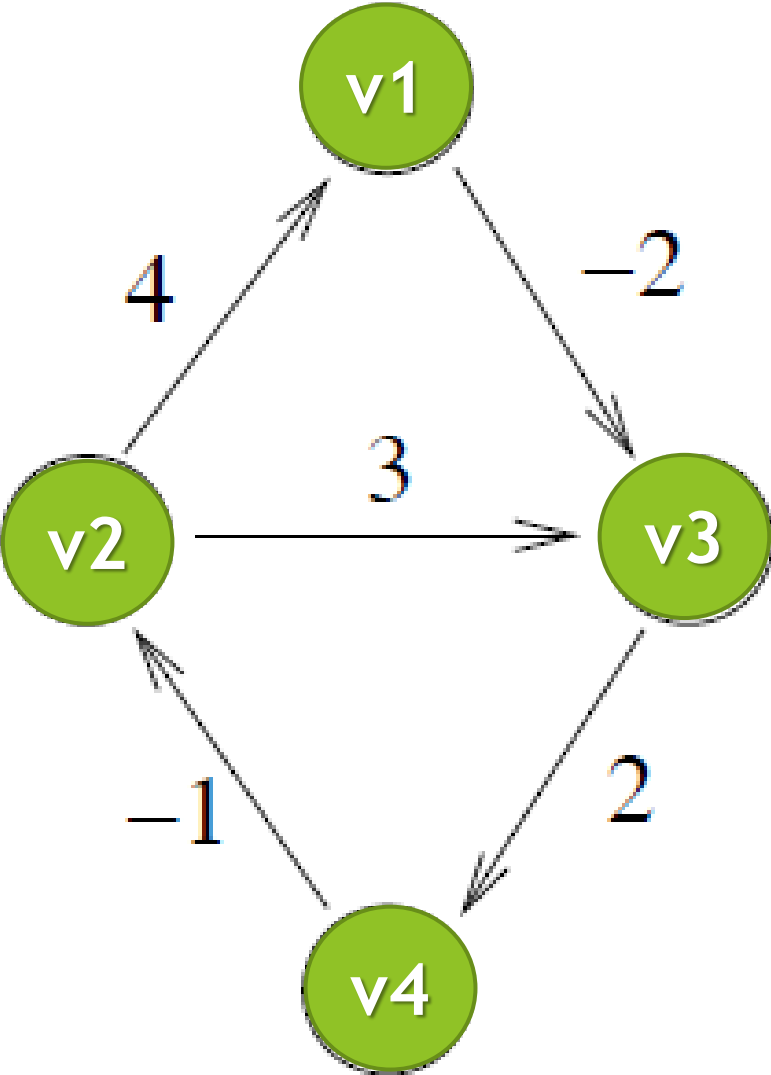


	v1	v2	v3	v4
v1	0	∞	-2	0
v2	4	0	2	4
v3	∞	∞	0	2
v4	3	-1	1	0

k= 3

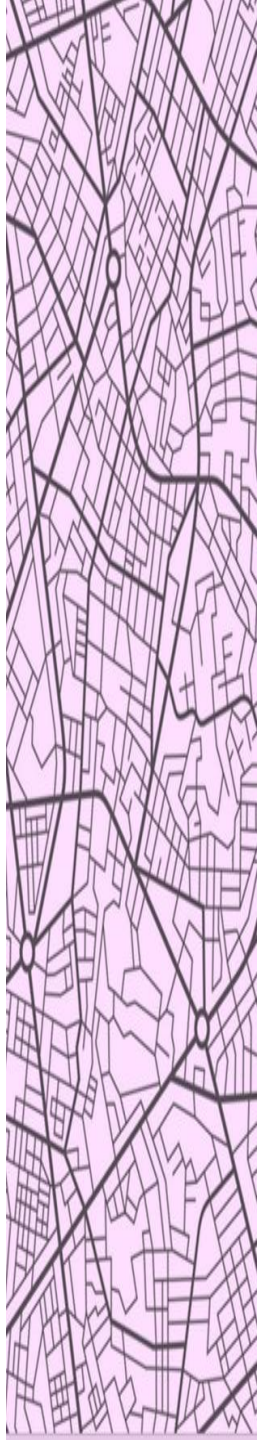


Algoritmo de Floyd -Warshall

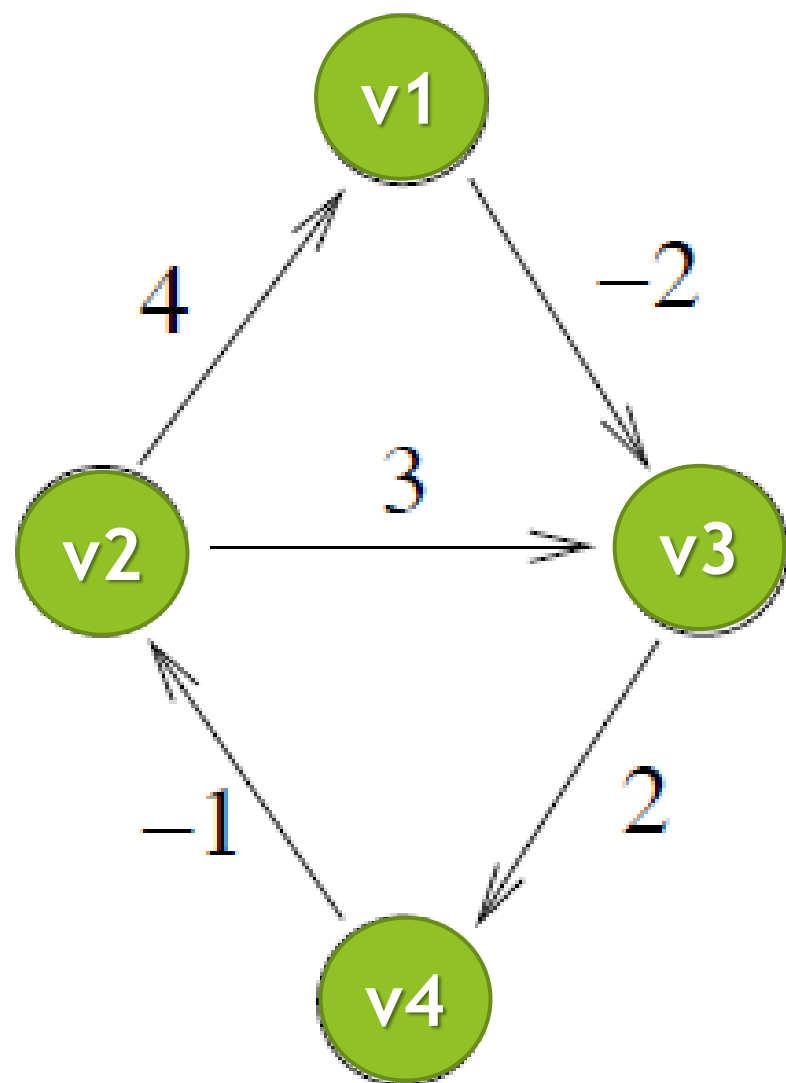


	v1	v2	v3	v4
v1	0	∞	-2	0
v2	4	0	2	4
v3	∞	∞	0	2
v4	3	-1	1	0

k= 4

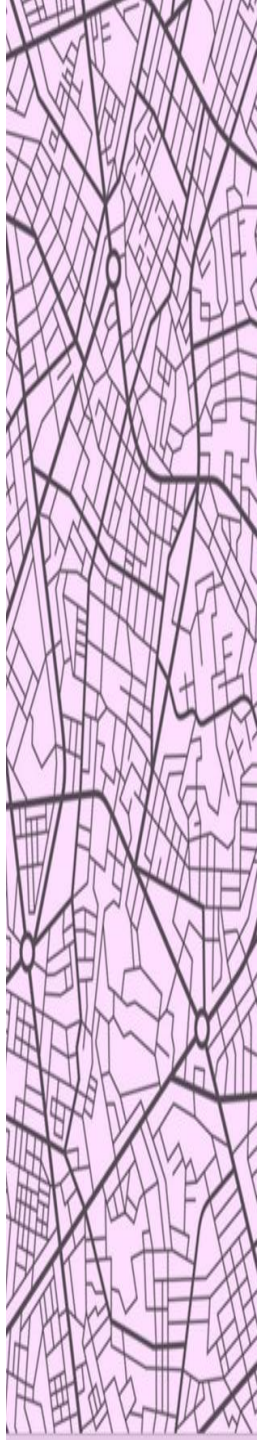


Algoritmo de Floyd -Warshall

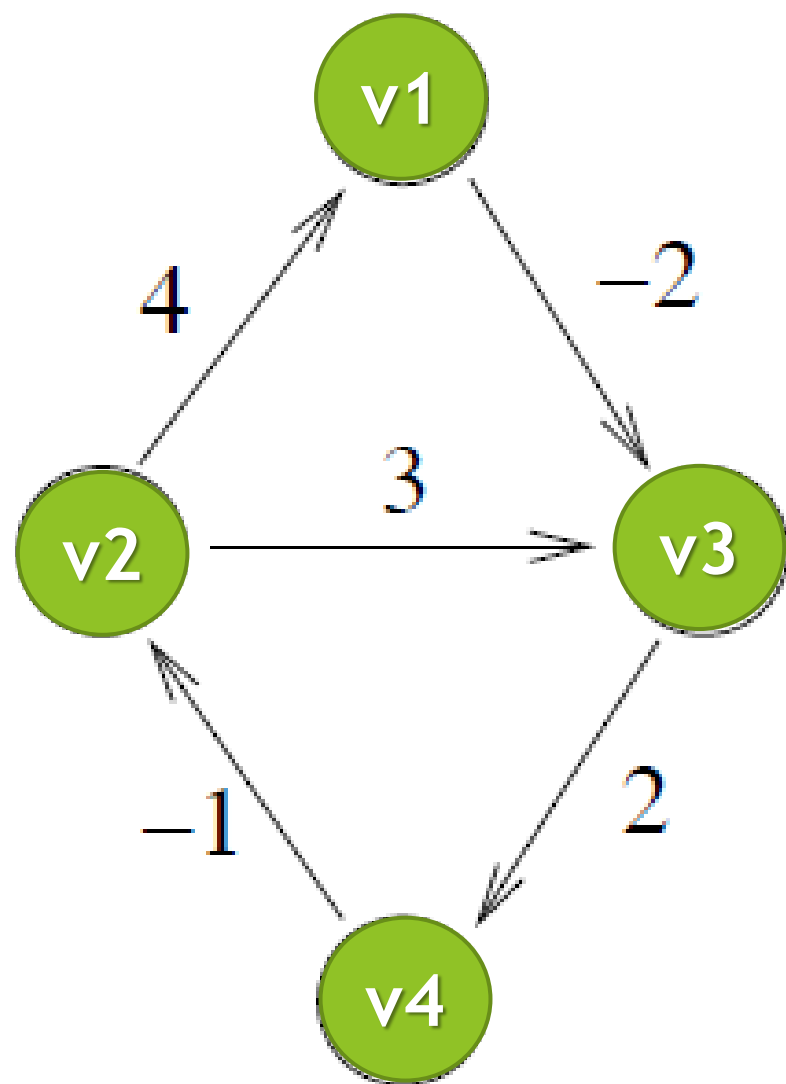


	v1	v2	v3	v4
v1	0	-1	-2	0
v2	4	0	2	4
v3	5	1	0	2
v4	3	-1	1	0

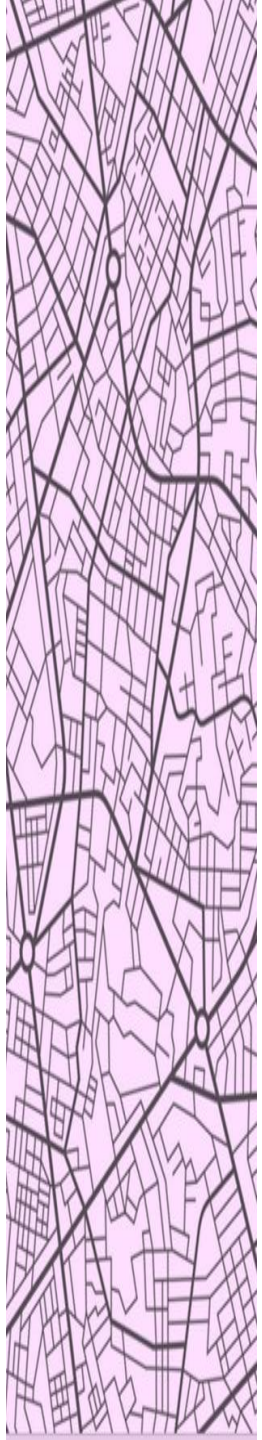
k= 4



Algoritmo de Floyd -Warshall



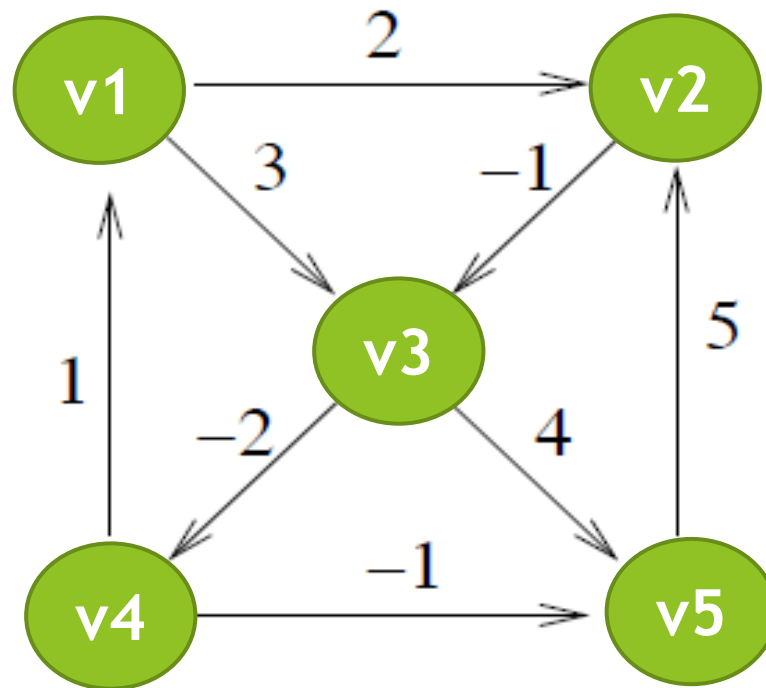
	v1	v2	v3	v4
v1	0	-1	-2	0
v2	4	0	2	4
v3	5	1	0	2
v4	3	-1	1	0



Algoritmo de Floyd -Warshall

Exercício

1. Execute o algoritmo de Floyd – Warshall para o grafo abaixo.



Algoritmo de Bellman-Ford-Moore

- Seja $D = (V, A)$ um **grafo dirigido**, possivelmente com **custos negativos** nas arestas, mas **sem ciclos negativos**, e seja u um vértice de D . O algoritmo de Bellman-Ford-Moore (1956, 1958) determina a distância de u aos demais vértices de D .
- Para cada vértice v mantemos uma distância tentativa $d(v)$ de u até v . No início $d(u) = 0$ e $d(v) = \infty$ para os demais vértices de D .



Algoritmo de Bellman-Ford-Moore

O algoritmo de Bellman-Ford-Moore pode ser resumido no seguinte laço iterativo:

Enquanto existir enlace (aresta) (v,w) tal que

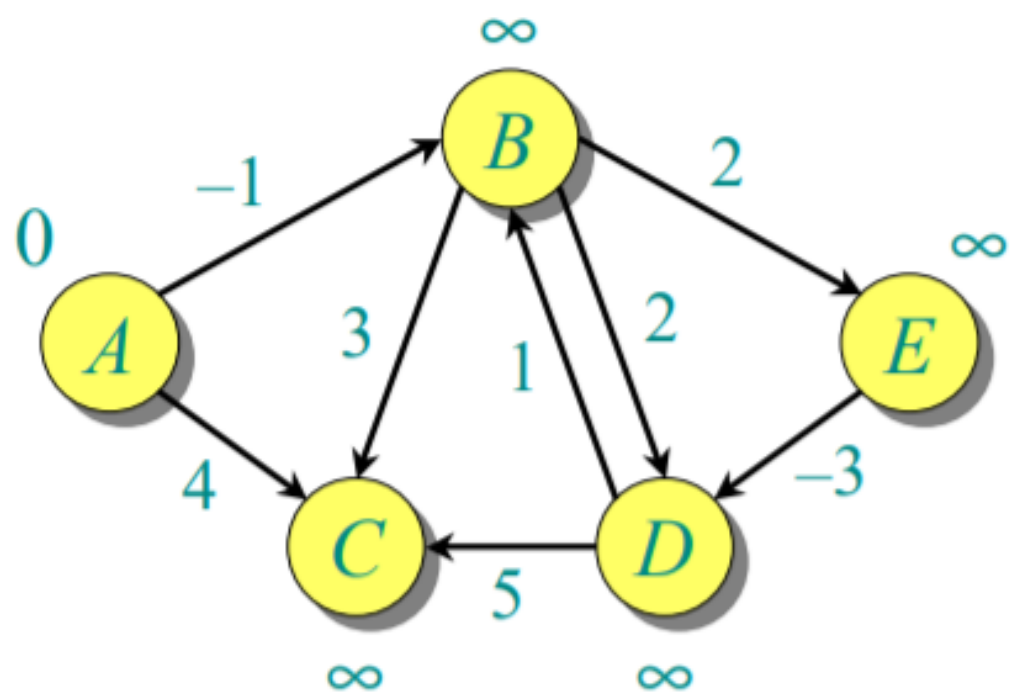
$d(w) > d(v) + c(v,w)$ faça

$$d(w) \leftarrow d(v) + c(v,w)$$



Example of Bellman-Ford

Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)

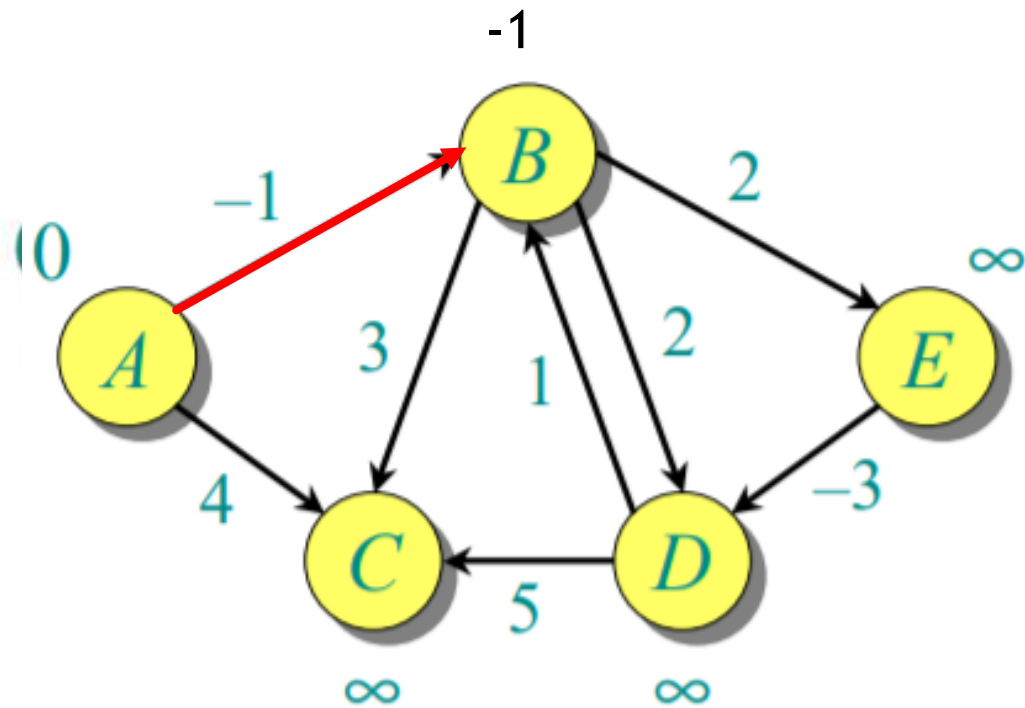


A	B	C	D	E
0	∞	∞	∞	∞



Example of Bellman-Ford

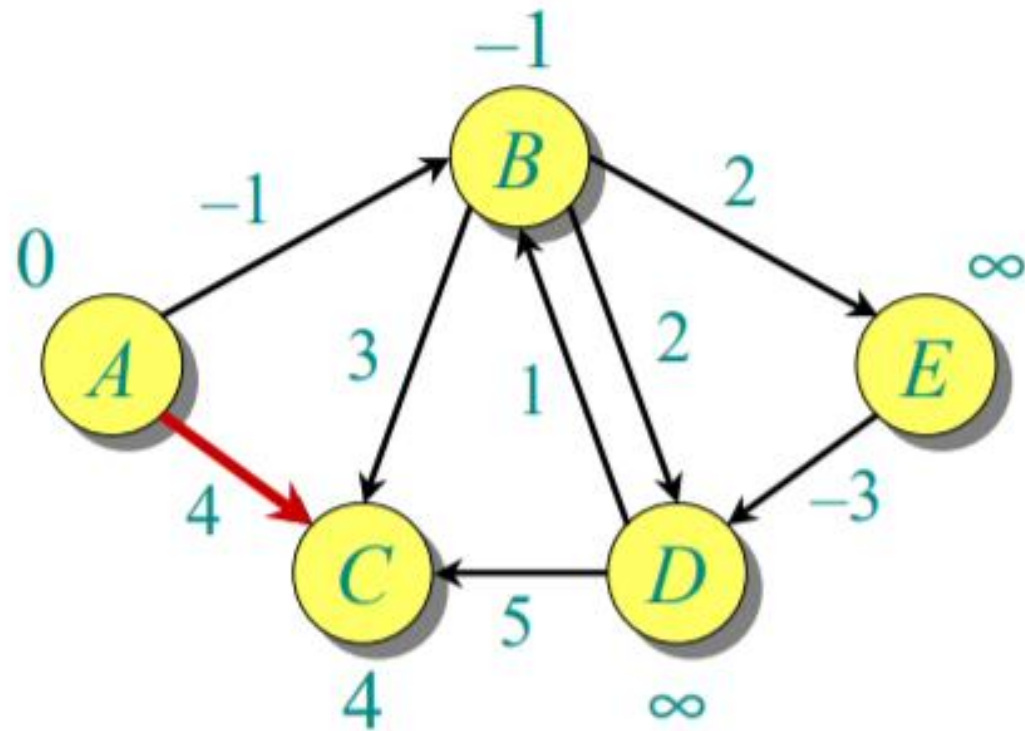
Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞

Example of Bellman-Ford

Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)

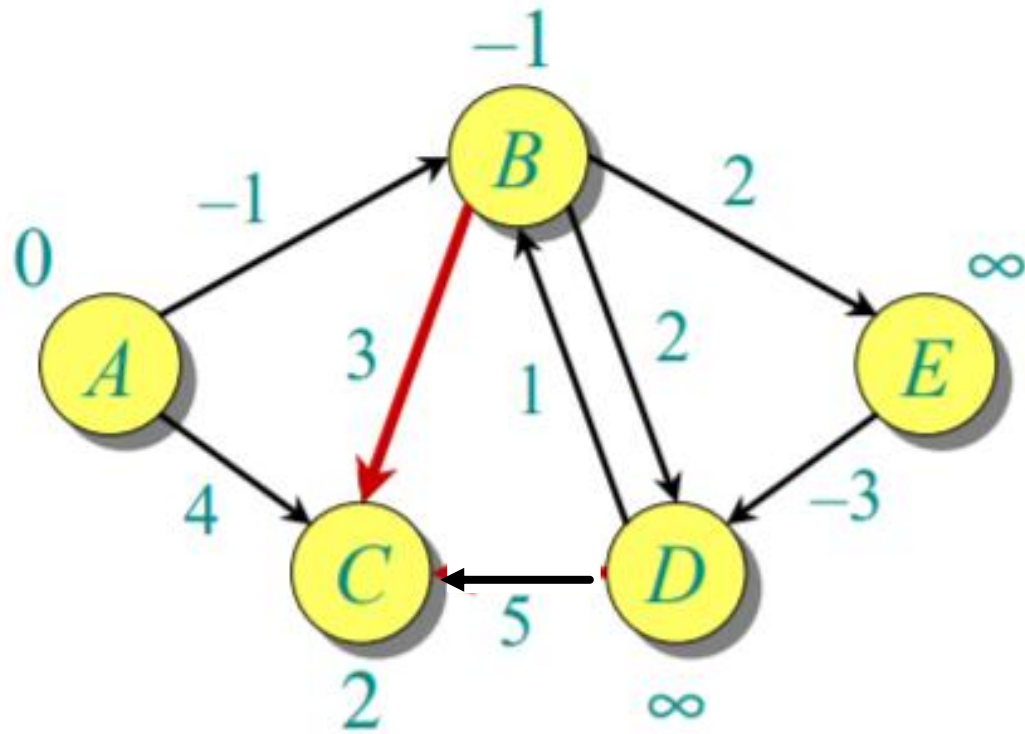


<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞



Example of Bellman-Ford

Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)

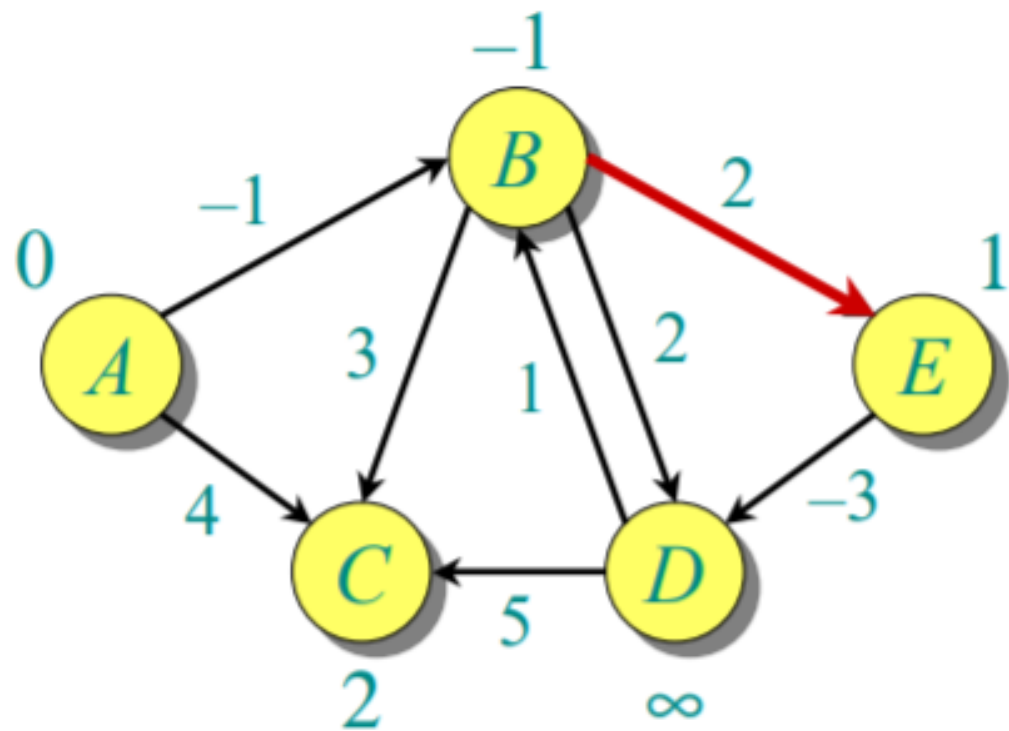


<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞



Example of Bellman-Ford

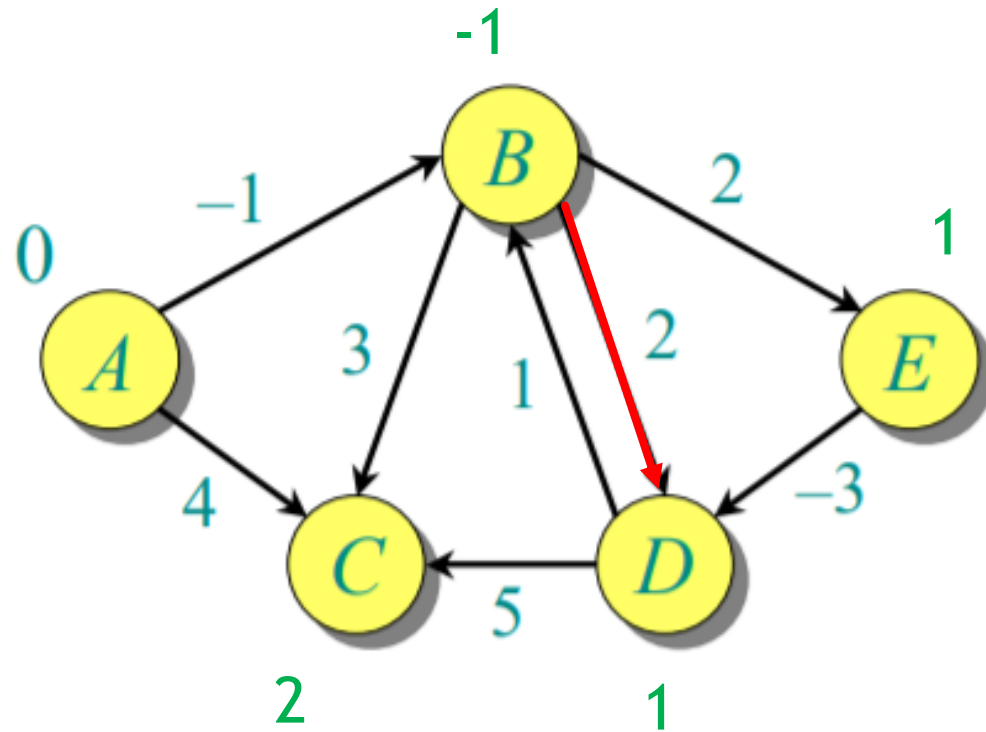
Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)



A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1

Example of Bellman-Ford

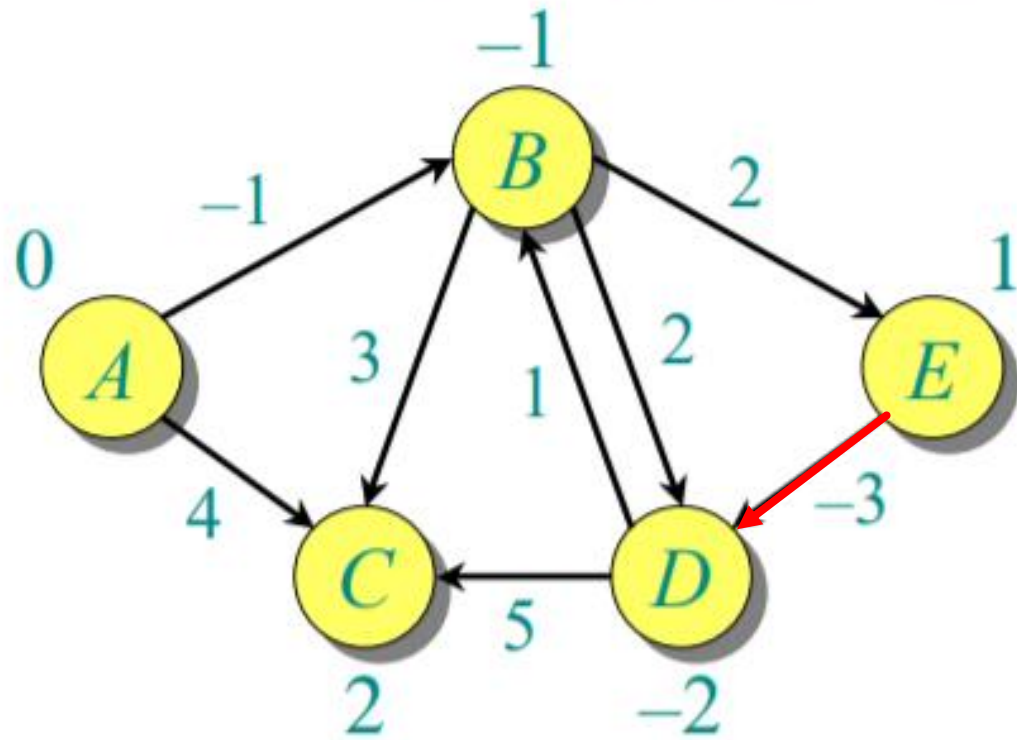
Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)



<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1

Example of Bellman-Ford

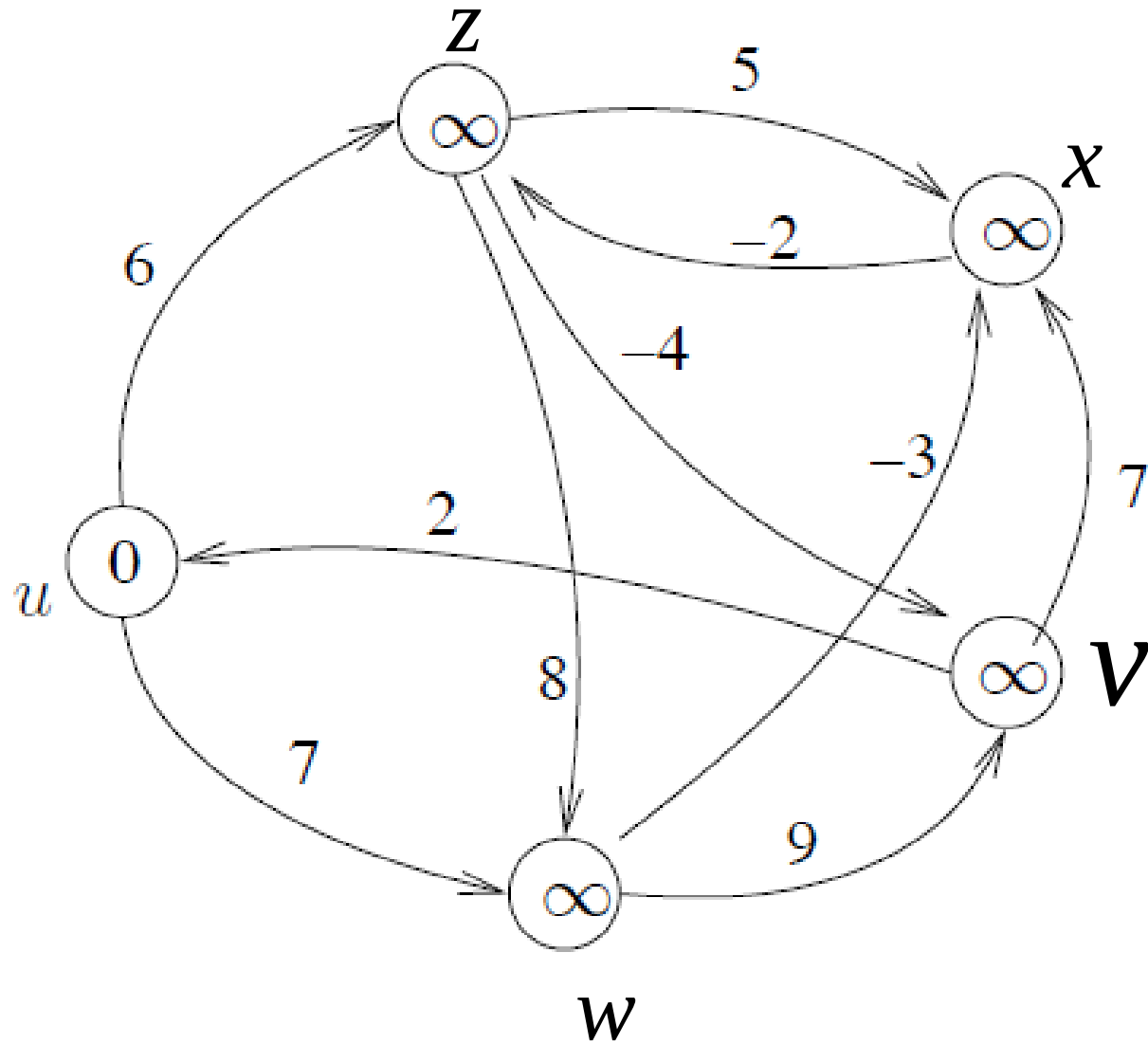
Order of edges: (B,E) , (D,B) , (B,D) , (A,B) , (A,C) , (D,C) , (B,C) , (E,D)



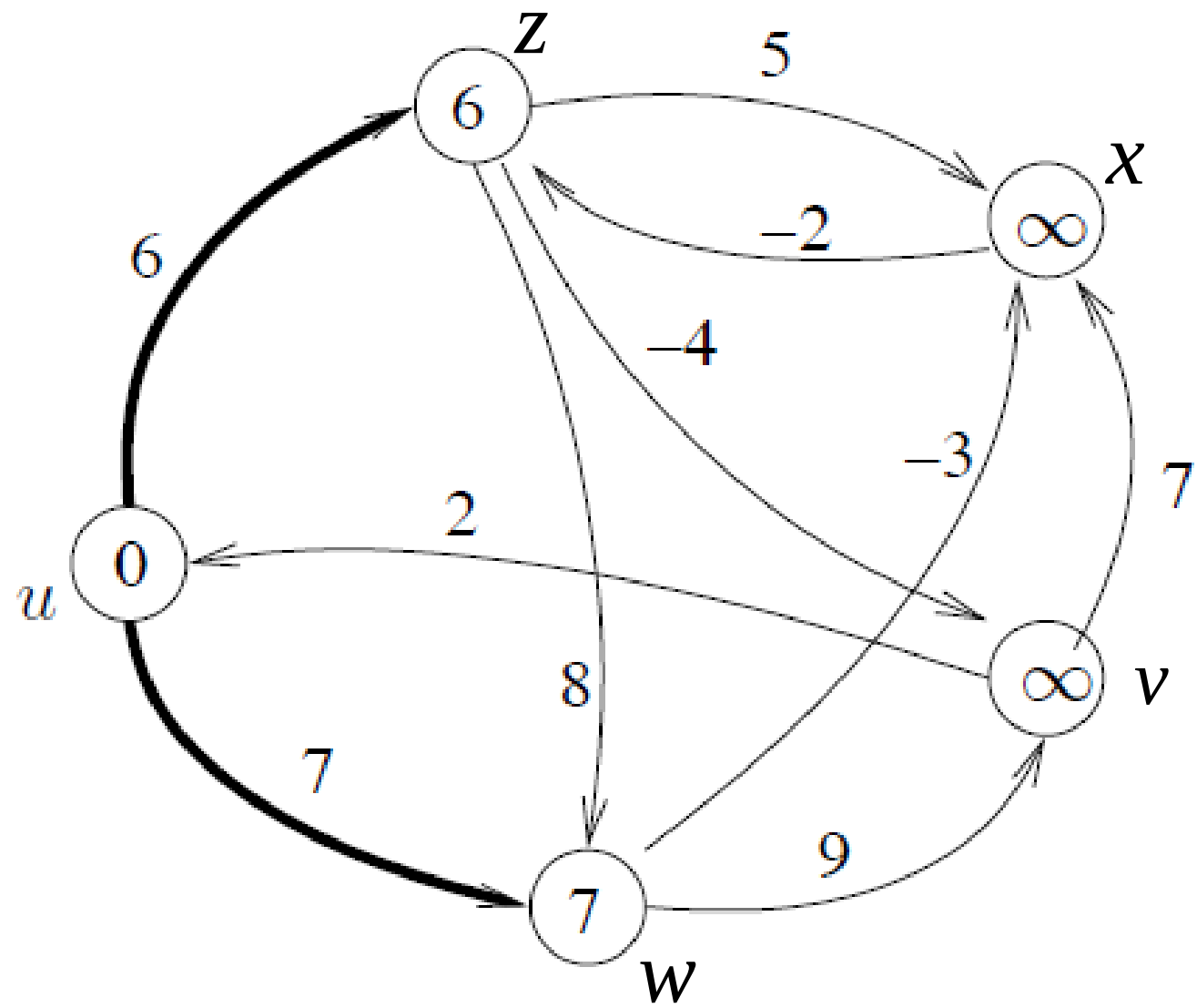
Note: Values decrease monotonically.

A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1
0	-1	2	-2	1

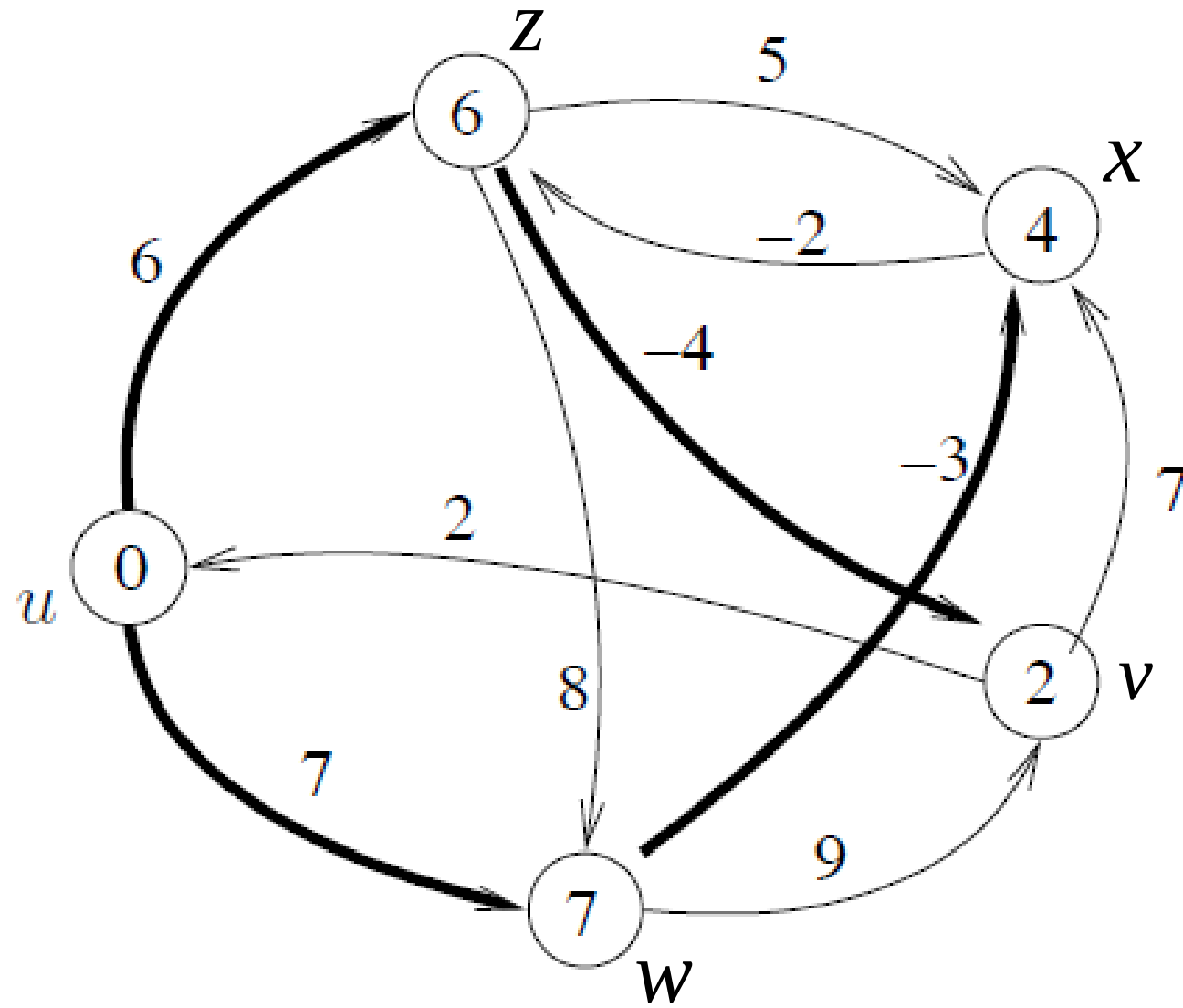
Algoritmo de Bellman-Ford-Moore



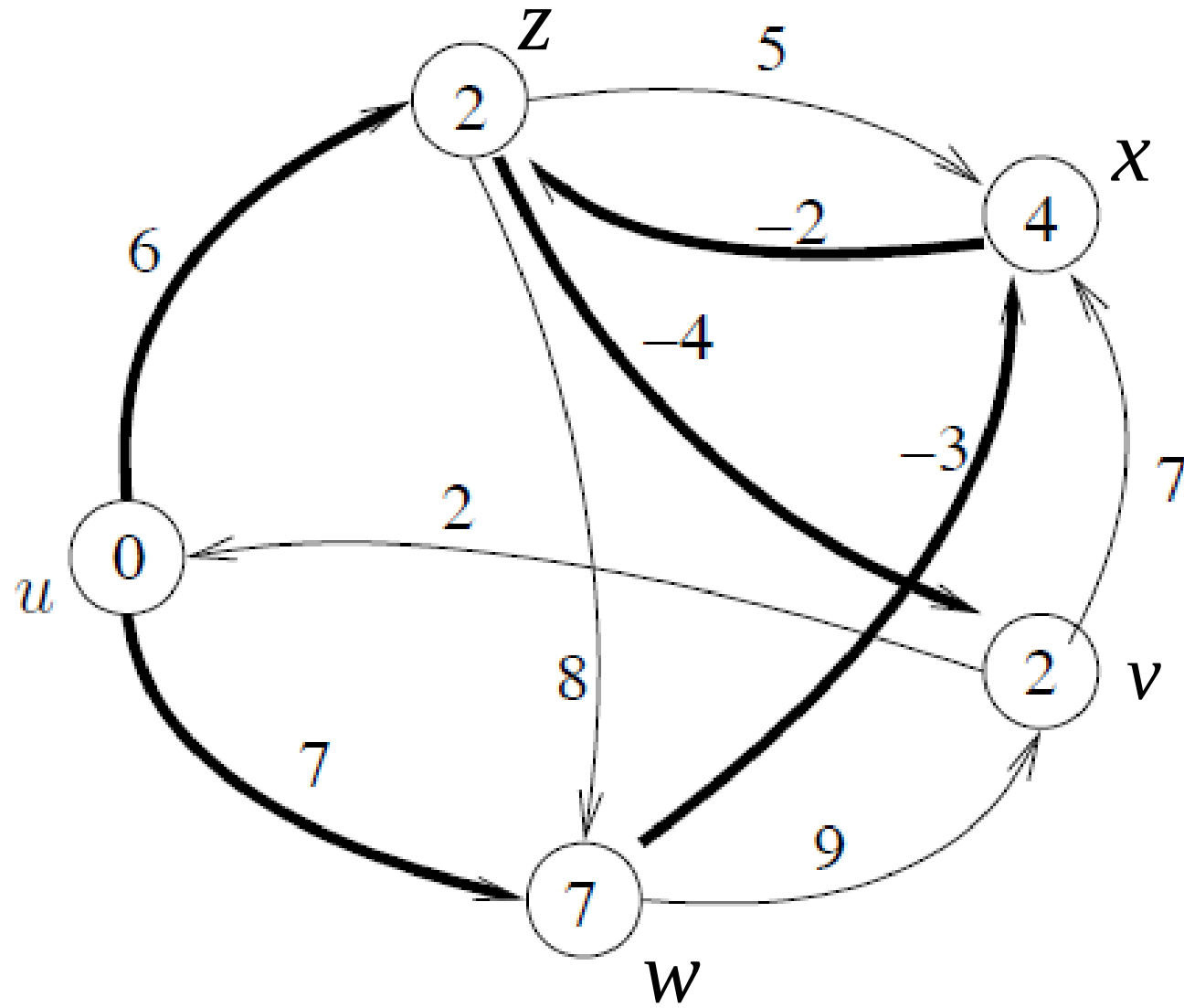
Algoritmo de Bellman-Ford-Moore



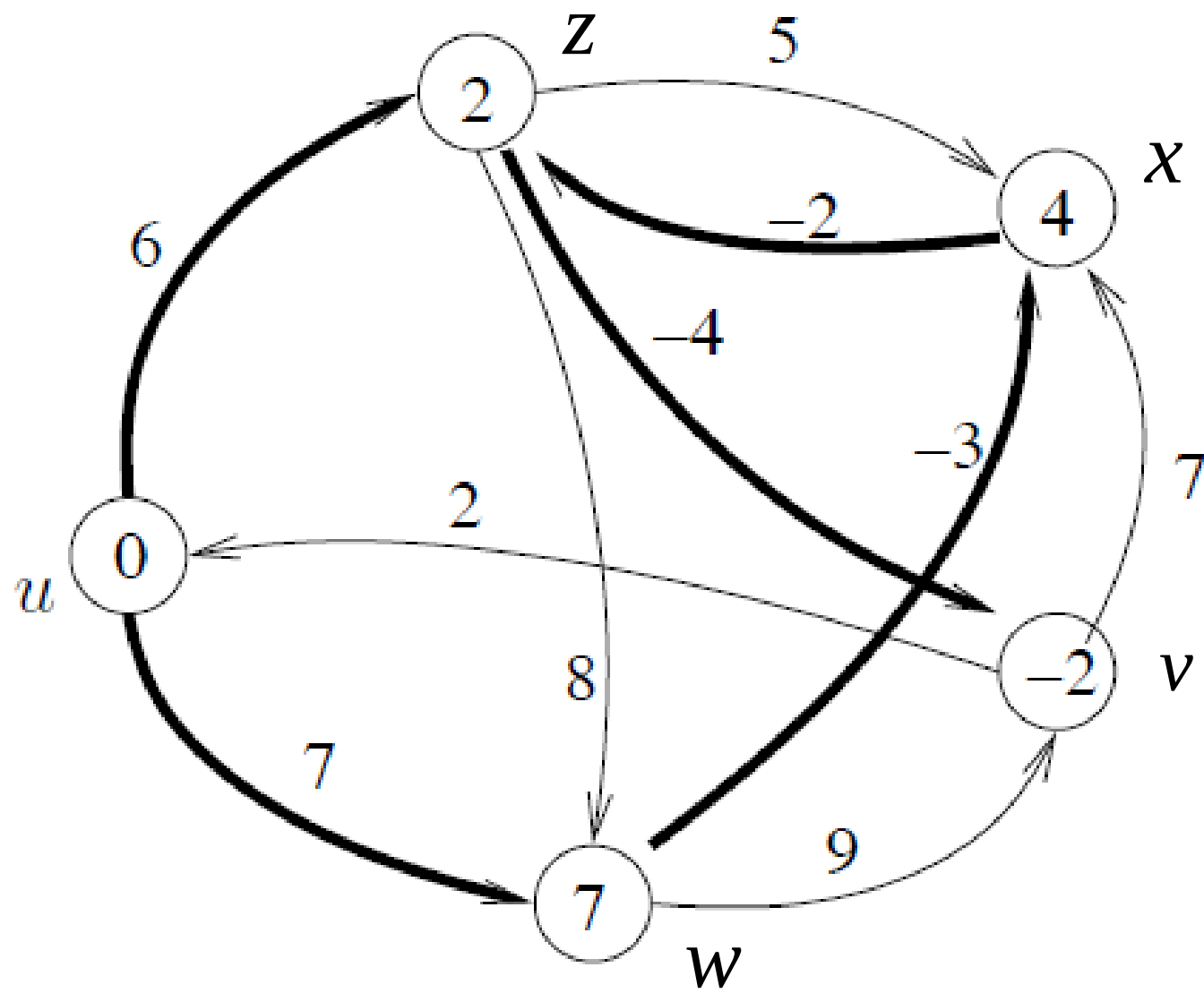
Algoritmo de Bellman-Ford-Moore



Algoritmo de Bellman-Ford-Moore



Algoritmo de Bellman-Ford-Moore



Algoritmo de Bellman-Ford-Moore

1. Para cada $v \in V(G)$ faça
 $d(v) \leftarrow \infty$
2. $d(u) \leftarrow 0$;
3. Para $i = 1$ até $|V(G)| - 1$ faça
Para cada arco $(v, w) \in E(G)$ faça
Se $d(w) > d(v) + c(v, w)$ então
 $d(w) \leftarrow d(v) + c(v, w)$
4. Para cada arco $(v, w) \in E(G)$ faça
Se $d(w) > d(v) + c(v, w)$ então
retorne(“impossível”)



Algoritmo de Bellman-Ford-Moore

- ▶ O Algoritmo de **Bellman-Ford** é um algoritmo de busca de **caminho mínimo em grafos dirigidos ponderado**, ou seja, cujas arestas têm peso, inclusive negativo.
- ▶ O algoritmo de **Dijkstra** resolve o mesmo problema, num **tempo menor**, porém exige que todas as arestas tenham pesos positivos.
- ▶ Portanto, o algoritmo de **Bellman-Ford** é normalmente usado apenas quando existem arestas de peso negativo.



Algoritmo de Bellman-Ford-Moore



Lester Randolph Ford Jr.



Richard Ernest Bellman

- Publicado em 1956 por Ford, em 1958 por Bellman e em 1957 por Edward F. Moore. Também conhecido como algoritmo de Bellman-Ford-Moore.
- Menos eficiente do que Dijkstra, mas trata arestas com pesos **negativos**. É capaz de detectar ciclos negativos.
- Complexidade $\mathcal{O}(nm)$



Algoritmo de Floyd -Warshall

Exercício

2. Execute o algoritmo de Floyd – Warshall para o grafo abaixo.

